

PlantUML时序图完全指南

ThinkStu 2023-10-17 5,644 阅读9分钟

关注

<<< TRAE 2.0 SOLO 出道，一键贯通从灵感火花到上线部署的全程协作 >>>

开源网站: [github.com/plantuml/pl...](https://github.com/plantuml/plantuml)
官方教学文档: plantuml.com/zh/sequence...

PlantUML 是一个开源工具，用于创建 UML（统一建模语言）图表。它让用户能够通过简单的文本描述来生成图表，而不是使用传统的图形编辑器。这种方法可以更快速、更有效地创建和维护图表。

PlantUML 支持多种 UML 图表类型，包括**类图**、**序列图**、**用例图**、**活动图**、**组件图**、**状态图**等。

一、基本例子

▼ shell

体验AI代码助手

复制代码

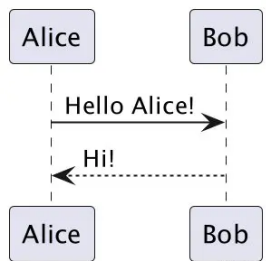
```
1 @startuml
2 participant Alice as A
3 participant Bob as B
4 A->>B: Hello Alice!
5 B-->>A: Hi!
6 @enduml
```

▼ shell

体验AI代码助手

复制代码

```
1 ' 单行注释
2 /' 多行注释 '/
```



二、基本语法

1、方向

- -> 实线
- --> 虚线

也存在 <- 与 <--，增加其可令可读性增高，不过较少使用。

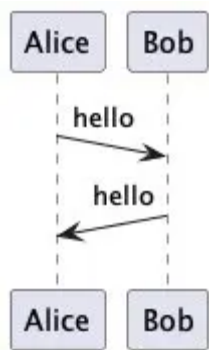
默认情况下，线条是平行的，我们可以使用 ->(倾斜度) 来使箭头倾斜：

▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 Alice ->(10) Bob : hello
3 Bob ->(10) Alice : hello
4 @enduml
```



默认情况下，箭头是以先后顺序依次向下递增，我们可以开启 PlantUML 的允许平齐配置，然后使用 & 使得部分箭头平齐：

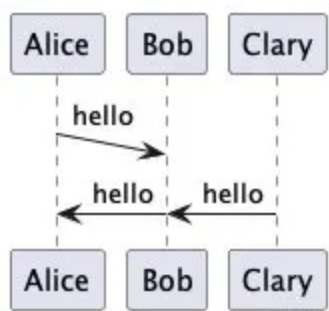
▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 !pragma teoz true
3 Alice ->(10) Bob : hello
4 Bob -> Alice : hello
```

```
5 & Clary -> Bob: hello
6 @enduml
```



2、参与者

在 PlantUML 中有多种角色可以表示，角色的声明顺序默认是其**显示顺序**。

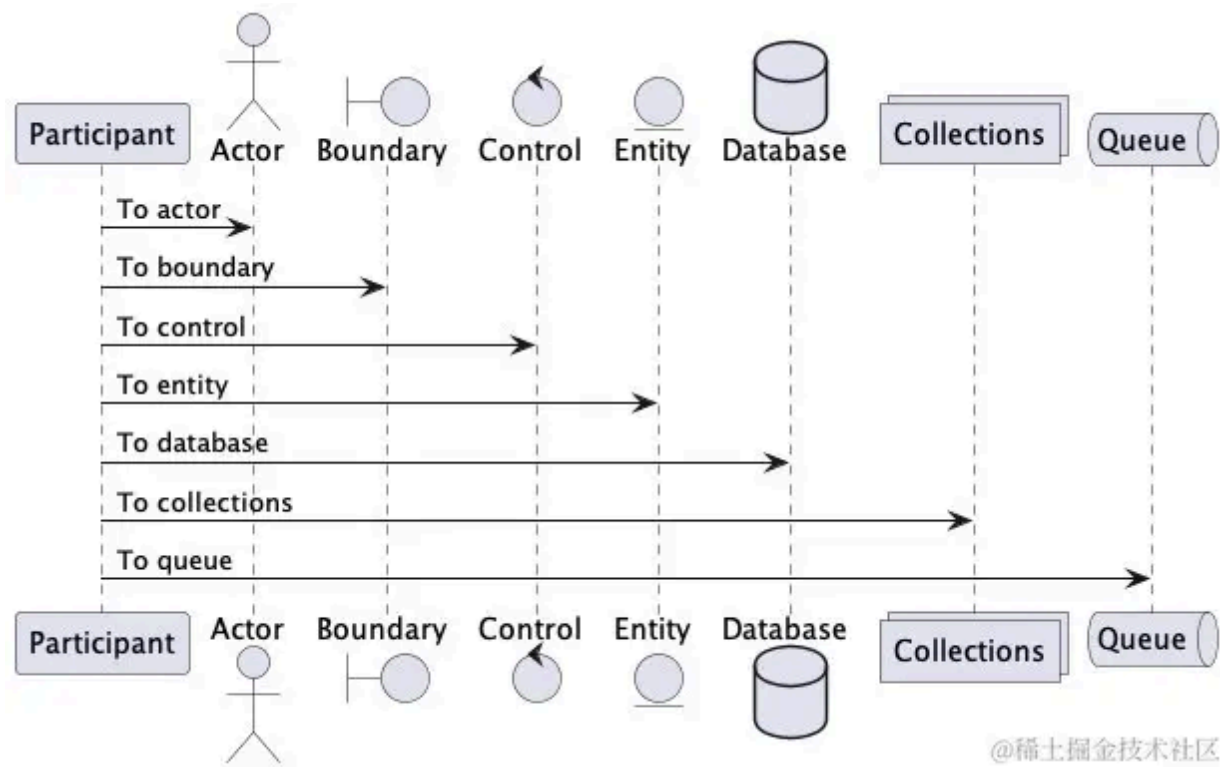
- participant: 普通参与者
- actor: 角色
- boundary: 边界
- control: 控制
- entity: 实体
- database: 数据库
- collection: 集合
- queue: 队列

▼ shell

 体验AI代码助手 

复制代码

```
1 @startuml
2 participant Participant as Foo
3 actor Actor as Foo1
4 boundary Boundary as Foo2
5 control Control as Foo3
6 entity Entity as Foo4
7 database Database as Foo5
8 collections Collections as Foo6
9 queue Queue as Foo7
10 Foo -> Foo1 : To actor
11 Foo -> Foo2 : To boundary
12 Foo -> Foo3 : To control
13 Foo -> Foo4 : To entity
14 Foo -> Foo5 : To database
15 Foo -> Foo6 : To collections
16 Foo -> Foo7: To queue
17 @enduml
```



我们可以使用 `as` 关键字重命名参与者，语法格式为：

▼ shell

体验AI代码助手

复制代码

```
1 participant Alice as A
```

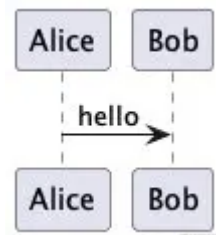
PlantUML 默认显示所有参与者，我们可以声明孤立未参与的参与者：

▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 hide unlinked
3 participant Alice
4 participant Bob
5 participant Carol
6
7 Alice -> Bob : hello
8 @enduml
```



`order` 关键字可以调整参与者的顺序，如：

▼ shell

 体验AI代码助手 

复制代码

```
1 'order 表示其排序的顺序。值越小，排序越靠前
2 actor Alice as A order 20
3 queue Bob as B    order 10
```

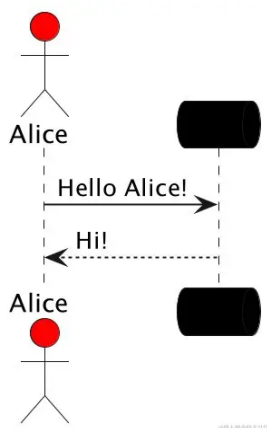
我们也可以修改参与者的背景色，如：

▼ shell

 体验AI代码助手 

复制代码

```
1 actor Alice as A #RED
2 queue Bob as B  #000
```



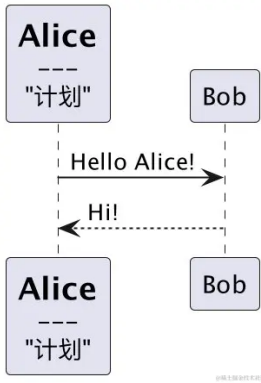
多行定义参与者：

▼ shell

 体验AI代码助手 

复制代码

```
1 @startuml
2 ' 原写法: participant Alice as A
3 participant A [
4     =Alice
5     ---
6     "计划"
7 ]
8 participant Bob as B
9 A->>B: Hello Alice!
10 B-->>A: Hi!
11 @enduml
```



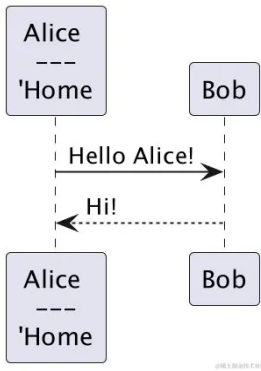
当定义的数据中存在“特殊”字符时，使用双引号包裹 `"`，可以使用 `\n` 换行转义符：

▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 participant "Alice \n---\n'Home" as A
3 participant Bob as B
4 A->>B: Hello Alice!
5 B-->>A: Hi!
6 @enduml
```



自己给自己发消息：

▼ shell

体验AI代码助手

复制代码

```
1 B->>B: hello
```

```
sequenceDiagram
    participant Bob as Bob
    Bob->>Bob: hello
```

3、skinparam初阶

改变图表样式和外观的选项，注意是 **skin** - param，而不是 skip - param

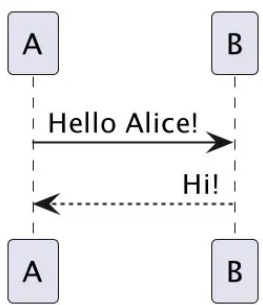
文本对齐： sequenceMessageAlign，后接参数 left、right 或 center

▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 skinparam SequenceMessageAlignment right
3 A->B: Hello Alice!
4 B-->A: Hi!
5 @enduml
```



4、箭头样式

- **-x**、**x->**、**x-x**、**x->o**：表示丢失的信息
- **-** 或 **-/**：只拥有箭头的上部分或下部分
- **->>** 或 **-**：拥有一个薄的箭头样式
- **--**：点状箭头样式
- **-o**：箭头末端会添加 **o**
- **<->**：双向箭头
- **-[#000]>**：修改箭头颜色（注意**颜色**在中间）

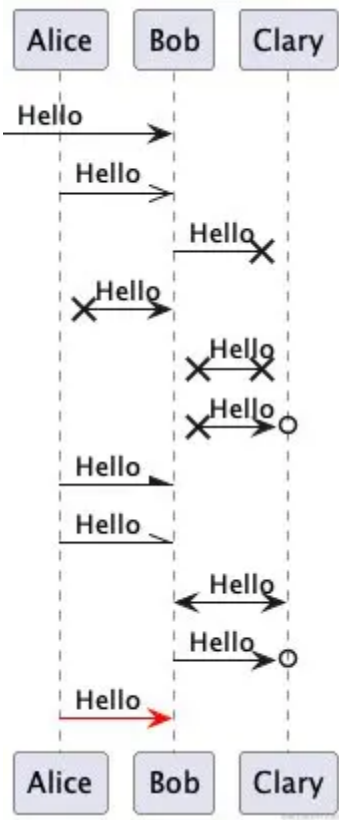
▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 participant "Alice" as A
3 participant Bob as B
4 participant Clary as C
5 A-> B: Hello
6 A->>B: Hello
7 B-x C: Hello
8 A x->B: Hello
9 B x-x C: Hello
10 B x->o C: Hello
11 A-\ B: Hello
```

```
12 A-\ B: Hello
13 B<->C: Hello
14 B->o C: Hello
15 A-[#red]> B: Hello
16 @enduml
```

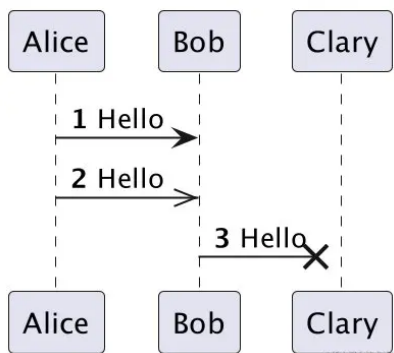


5、序号

我们可以使用 `autonumber` 对消息进行自动编号

```
▼ shell
```

```
1 @startuml
2 participant "Alice" as A
3 participant Bob as B
4 participant Clary as C
5 autonumber
6 A-> B: Hello
7 A->>B: Hello
8 B-x C: Hello
9 @enduml
```

- **autonumber 数字**：指定开始的初始序号
- **autonumber "[<u>0</u>]"**：指定数字格式，格式是由 Java 的 `DecimalFormat` 类实现的，(`0` 表示数字； `#` 也表示数字，但默认为0)。
- **autonumber stop**、**autonumber resume**：暂停、恢复计数
- **autonumber 1.1.1**：使用**分隔符**定义数字格式，如 `.`、`,`、`;`、`:` 等，**最后一位数字会自动递增**。要增加第一个数字，使用 **autonumber inc A**，第二个则为 **B**，以此类推。另外每次调用都会刷新比它等级更低的数值，默认为1。
- **%autonumber%**：使用 `autonumber` 变量的值。

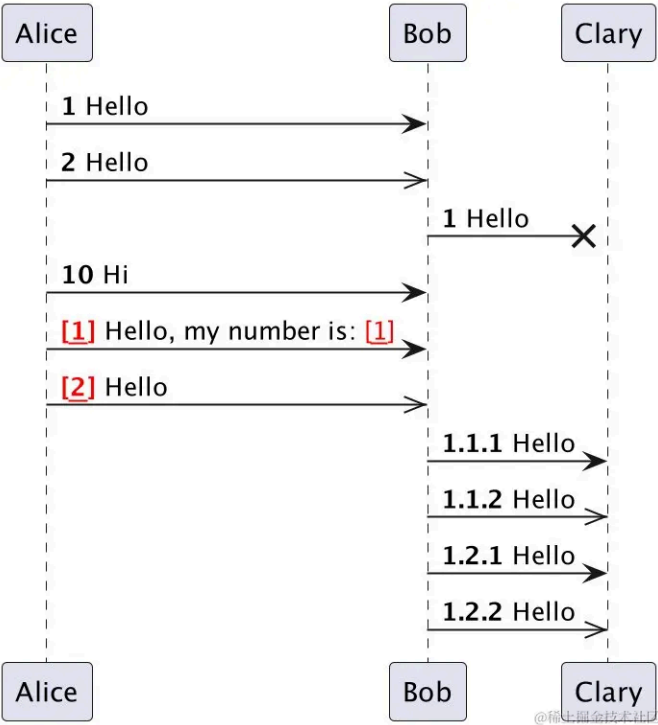
▼ shell

[体验AI代码助手](#)
[复制代码](#)

```

1 @startuml
2 participant "Alice" as A
3 participant Bob as B
4 participant Clary as C
5 autonumber
6 A-> B: Hello
7 A->>B: Hello
8 autonumber
9 B-x C: Hello
10 autonumber 10
11 A->B: Hi
12 autonumber "<font color=red><b>[<u>0</u>]</b>"
13 A-> B: Hello, my number is: %autonumber%
14 A->>B: Hello
15 autonumber 1.1.1
16 B-> C: Hello
17 B->>C: Hello
18 autonumber inc B
19 B-> C: Hello
20 B->>C: Hello
21 @enduml

```



6、页面配置

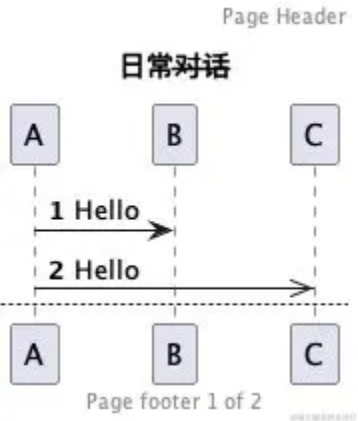
- 页面标题/页眉/页脚，其中标题支持 creole 格式
- 新页面： `newpage` , 可以同时携带上新标题

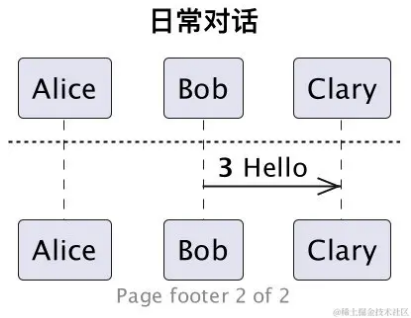
▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 title "日常--对话--"
3 header "Page Header"
4 footer "Page footer %page% of %lastpage%"
5 autonumber
6 A-> B: Hello
7 A->>C: Hello
8 newpage 日常对话
9 B->>C: Hello
10 @enduml
```





我们也可以使用 `hide footbox` 显式声明移除页脚（脚注）

▼ shell

体验AI代码助手 复制代码

```
1 @startuml
2
3 hide footbox
4 title Footer removed
5
6 Alice -> Bob: Authentication Request
7 Bob --> Alice: Authentication Response
8
9 @enduml
```

7、分组

我们可以通过以下关键词来组合消息，分组可以嵌套、关键词 `==end==` 结束分组：

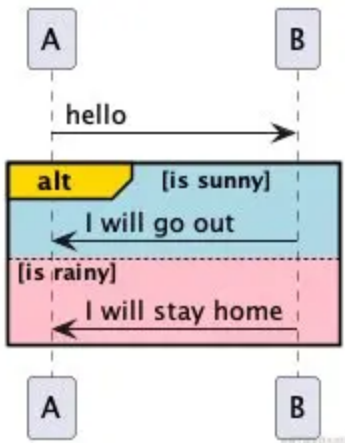
- `alt/else`：用于条件执行，如果条件满足则执行 `alt` 块中的内容，否则 `else`
- `opt`：optional，可选的选项，只有当某个条件满足时才会被触发
- `loop`：循环执行块
- `par`：parallel，并行执行块
- `break`：中断循环或其他结构
- `critical`：关键块，强调某个操作的重要性
- `group`：创建一个带有命名的分组
- `ref`：引用

我们通过左边顶端显示的英文信息，判断该组合框里代表什么消息类型，也可以为分组的标题框与内容框着色。

▼ shell

体验AI代码助手 复制代码

```
1 @startuml
2 A -> B: hello
3 alt#Gold #Lightblue is sunny
4     B->A: I will go out
5 else #Pink is rainy
6     B->A: I will stay home
7 end
8 @enduml
```

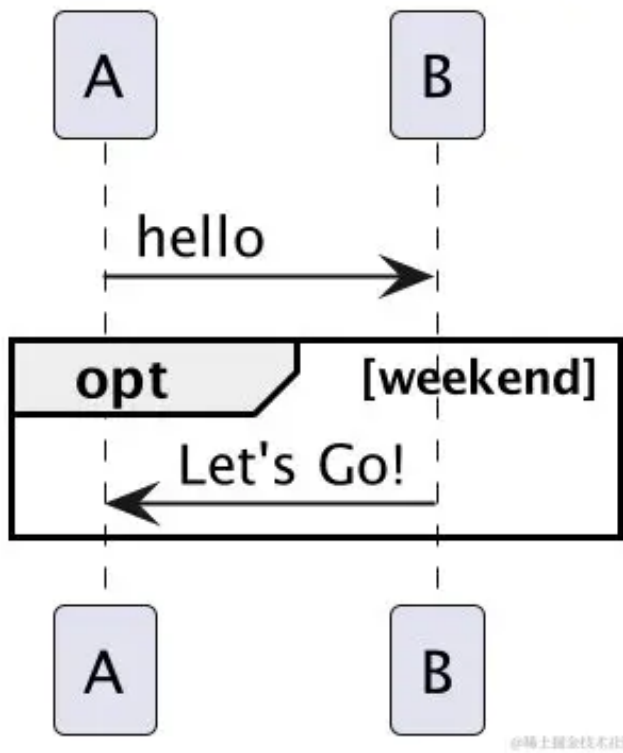


▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 A -> B: hello
3 opt weekend
4     B->A: Let's Go!
5 end
6 @enduml
```

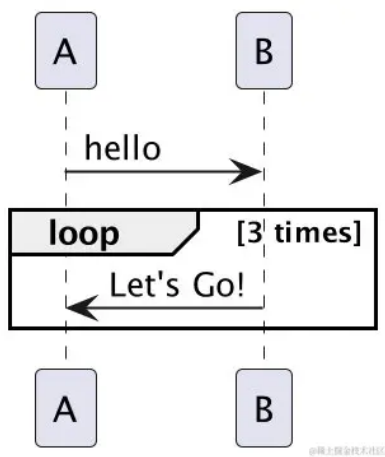


▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 A -> B: hello
3 loop 3 times
4     B->A: Let's Go!
5 end
6 @enduml
```



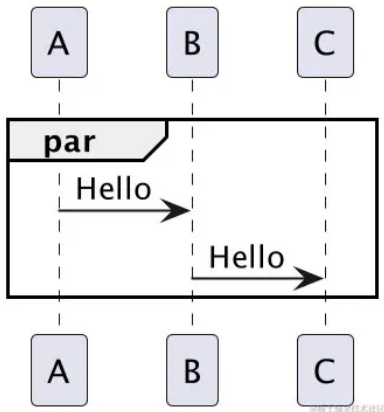
▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 par
3     A->B: Hello
4     B->C: Hello
```

```
5 end
6 @enduml
```



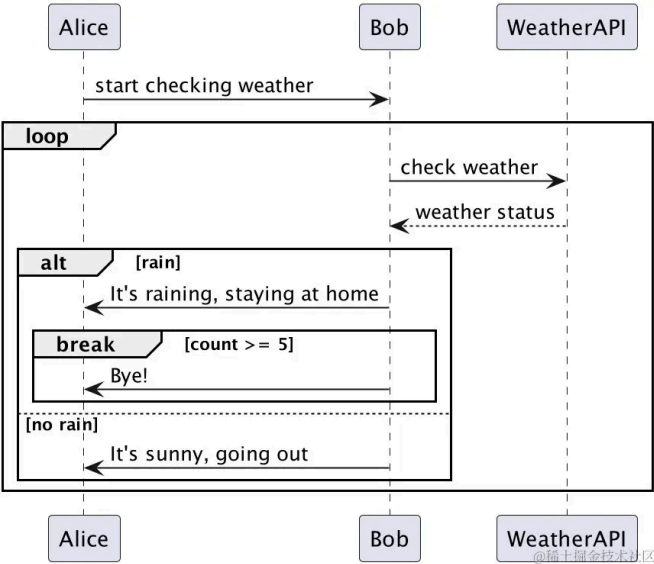
注意：break 需要 end 对应，中间可插入其他操作。

▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 Alice -> Bob : start checking weather
3 loop
4     Bob -> WeatherAPI : check weather
5     WeatherAPI --> Bob : weather status
6     alt rain
7         Bob -> Alice : It's raining, staying at home
8         break count >= 5
9         Bob -> Alice : Bye!
10    end
11    else no rain
12        Bob -> Alice : It's sunny, going out
13    end
14 end
15 @enduml
```

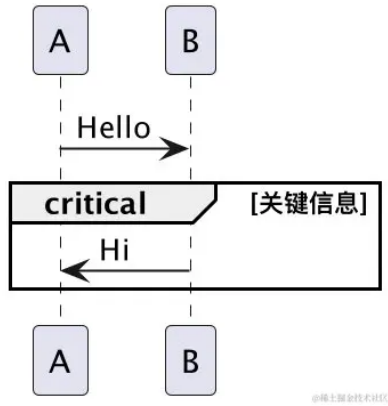


▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 A->>B: Hello
3 critical 关键信息
4     B->>A: Hi
5 end
6 @enduml
```

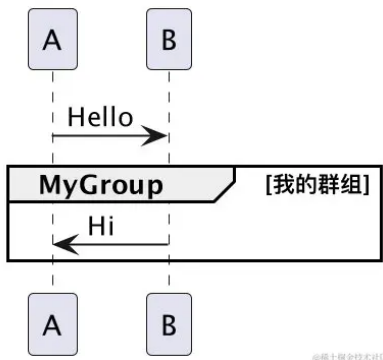


▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 A->>B: Hello
3 group MyGroup [我的群组]
4     B->>A: Hi
5 end
6 @enduml
```



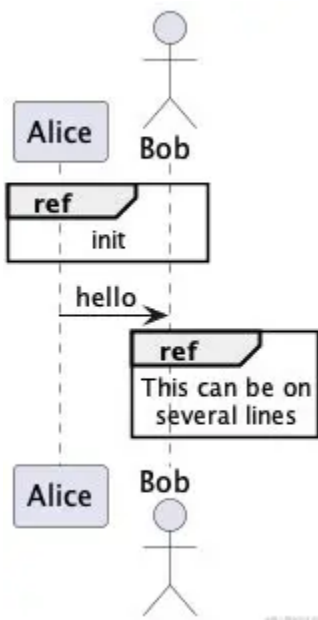
@稀土掘金技术社区

▼ shell

[体验AI代码助手](#)
[复制代码](#)

```

1  @startuml
2  participant Alice
3  actor Bob
4
5  ref over Alice, Bob : init
6
7  Alice -> Bob : hello
8
9  ref over Bob
10   This can be on
11   several lines
12 end ref
13 @enduml
  
```



@稀土掘金技术社区

8、备注信息

使用关键字 `note left` 或 `note right`，然后在后面加上「对象」和「信息」。当需要多行备注时，可以使用 `end note` 结束备注。

可以使用 `note left` 对象 或 `note left of` 对象、`note over` 对象 来在对应的地方释放备注。也可以在使用的过程中更改注释卡片的颜色，甚至改变形状：

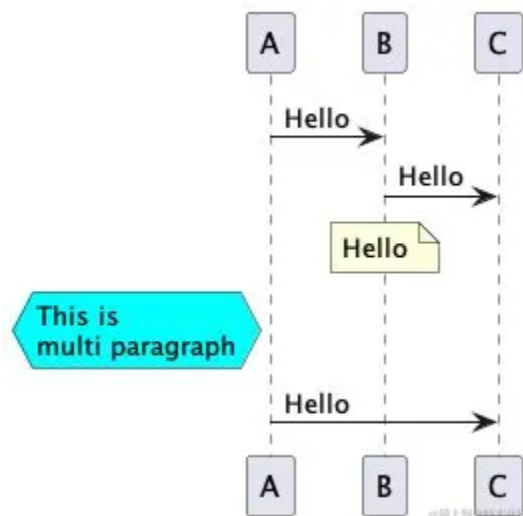
- `hnote`: hexagonal 六边形
- `rnote`: rectangle 正方形

▼ shell

 体验AI代码助手 

复制代码

```
1 @startuml
2     A->B: Hello
3     B->C: Hello
4     ' 单行信息
5     note over B: Hello
6     ' 多行信息
7     hnote left A #aqua
8     This is
9     multi paragraph
10 end note
11     A->C: Hello
12 @enduml
```



如果要在同一级对齐多个备注，使用 `/`：

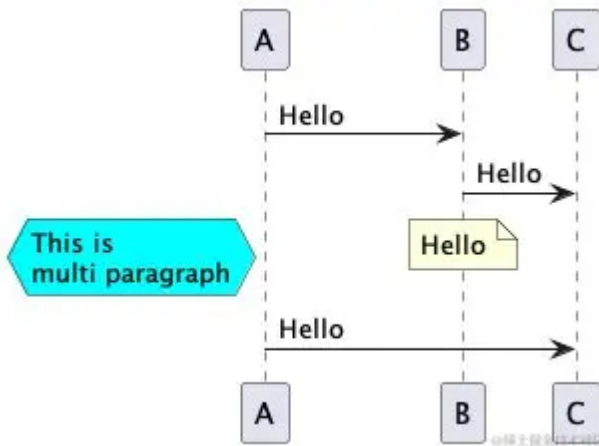
▼ shell

 体验AI代码助手 

复制代码

```
1 @startuml
2     A->B: Hello
3     B->C: Hello
4     ' 单行信息
5     note over B: Hello
6     ' 多行信息
7     / hnote left A #aqua
8     This is
```

```
9 multi paragraph
10 end note
11     A->>C: Hello
12 @enduml
```



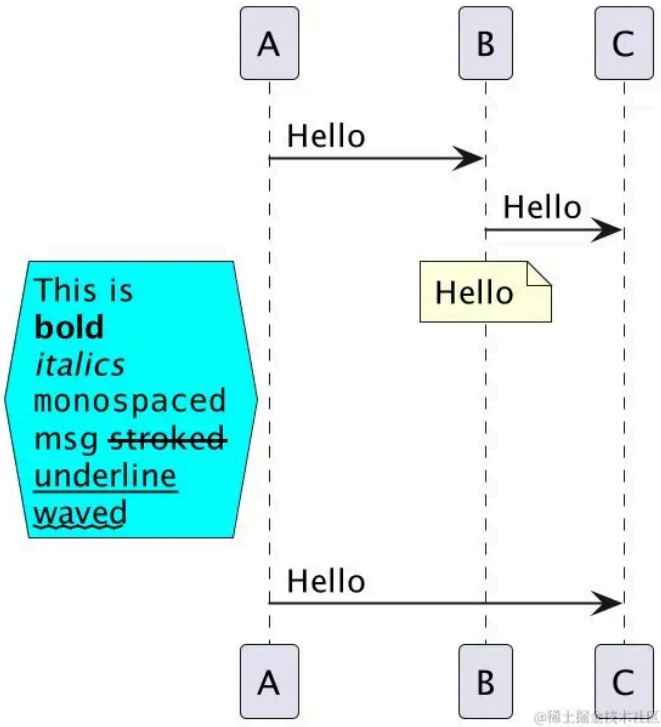
多行文本支持 [Creole](#) 和 HTML:

▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2     A->>B: Hello
3     B->>C: Hello
4     ' 多行信息
5     / hnote left A #aqua
6     This is
7     bold
8     //italics//
9     "monospaced"
10    msg --stroked--
11    __underline__
12    ~~waved~~
13 end note
14     A->>C: Hello
15 @enduml
```



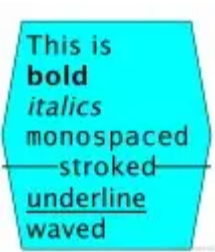
注意，`--` 需要前面带有其他多余字符时才生效，否则不符合预期

▼ shell

体验AI代码助手

复制代码

```
1  --stroked--
```



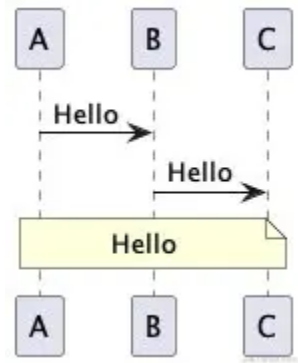
可以通过 `note across` 便捷的覆盖全部参与者

▼ shell

体验AI代码助手

复制代码

```
1  @startuml
2      A->>B: Hello
3      B->>C: Hello
4      note across: Hello
5      @enduml
```



9、分隔符

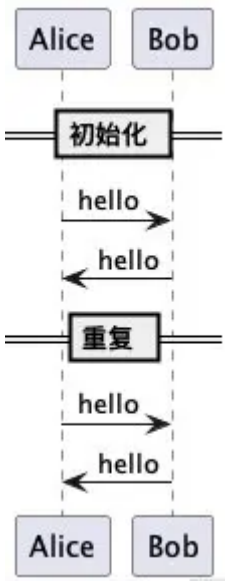
== 关键字

▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 participant Alice
3 participant Bob
4 == 初始化 ==
5 Alice -> Bob : hello
6 Bob -> Alice : hello
7 == 重复 ==
8 Alice -> Bob : hello
9 Bob -> Alice : hello
10 @enduml
```



10、延迟

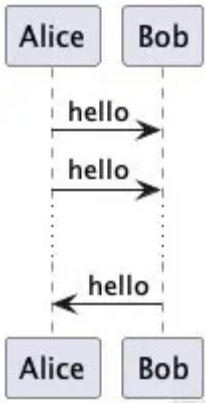
表示“延迟” ... ， 间距会变大，可在某间隔处多次使用。

▼ shell

 体验AI代码助手 

复制代码

```
1 @startuml
2 participant Alice
3 participant Bob
4 Alice -> Bob : hello
5 Alice -> Bob : hello
6 ...
7 Bob -> Alice : hello
8 @enduml
```



另外使用 `|||`（空间）也可以表示出相似的效果，但是 `|||`有新写法 `||像素数||`。

▼ shell

 体验AI代码助手 

复制代码

```
1 Alice -> Bob : hello
2 ||50||
3 Bob -> Alice : hello
```

11、生命线

在 PlantUML 顺序图中，关键字 `activate` 和 `deactivate` 用于表示参与者的活动状态。当一个参与者被激活时，其生命线会显示为一个较宽的矩形。这通常表示参与者正在执行某个操作或处理某个任务。当参与者变为非激活状态时，生命线会恢复为正常宽度。

关键字 `destroy` 用于表示参与者的生命周期结束。当一个参与者被销毁时，其生命线会显示为一个带有  的矩形。另外，生命线条可以添加颜色。

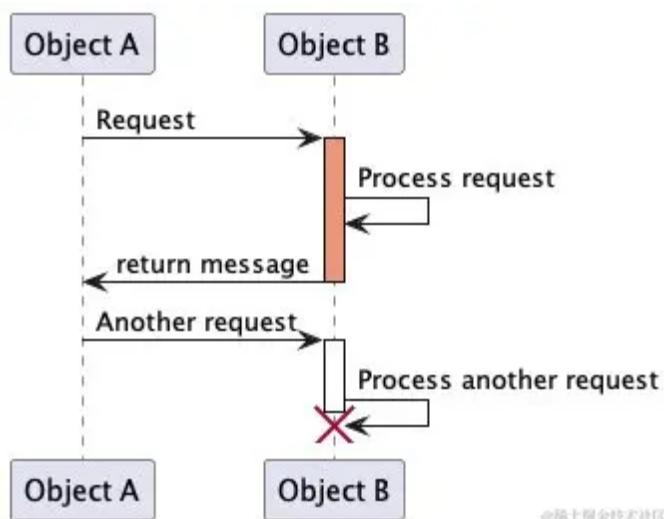
▼ shell

 体验AI代码助手 

复制代码

```
1 @startuml
2 participant "Object A" as A
3 participant "Object B" as B
```

```
4 A->>B: Request
5 activate B #DarkSalmon
6 B->>B: Process request
7 B->>A: return message
8 deactivate B
9 A->>B: Another request
10 activate B
11 B->>B: Process another request
12 destroy B
13 @enduml
```



©稀土掘金技术社区

在这个示例中，当 Object A 向 Object B 发送请求时，Object B 被激活并处理请求。处理完成后返回消息，Object B 变为非激活状态。随后，Object A 发送另一个请求，Object B 再次被激活并处理请求。最后，Object B 的生命周期结束，生命线显示为带有 **X** 的矩形。

激活、撤销、创建的快捷语法：

在指定目标参与者后，可以立即使用以下语法：

- **++** 激活目标（可选择在后面加上[#color]）
- **--** 撤销激活源
- ****** 创建目标实例
- **!!** 摧毁目标实例

▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 participant "Object A" as A
3 participant "Object B" as B
4 A->>B: Request
5 B++ #DarkSalmon
```

```
6 B->>B: Process request
7 B-->A: return message
8 B--
9 A->>B: Another request
10 B++ #DarkSalmon
11 B->>B: Process another request
12 @enduml
```

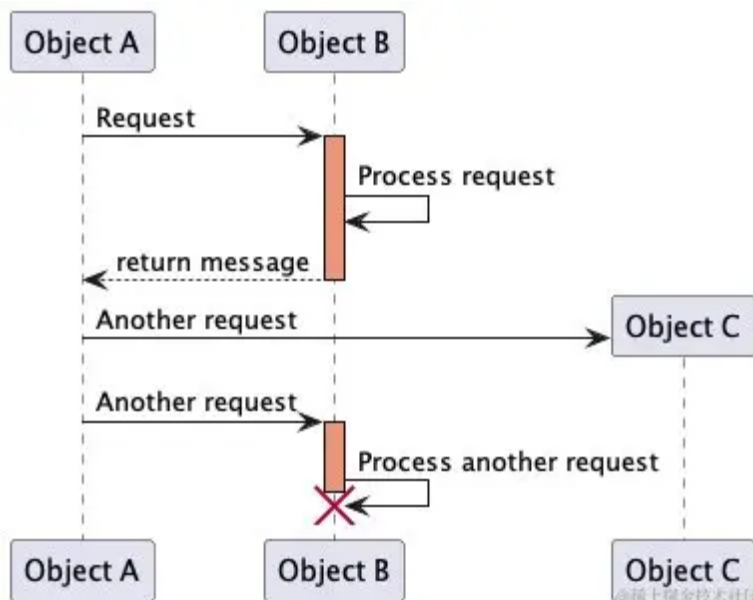
或者可以使用下面这种写法：

▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 participant "Object A" as A
3 participant "Object B" as B
4 A->>B ++ #DarkSalmon: Request
5 B->>B: Process request
6 B-->A --: return message
7 A->>"Object C" ** #DarkSalmon: Another request
8 A->>B ++ #DarkSalmon: Another request
9 B->>B !!: Process another request
10 @enduml
```



12、return

自动返回，返回的依据是上一个用例。

▼ shell

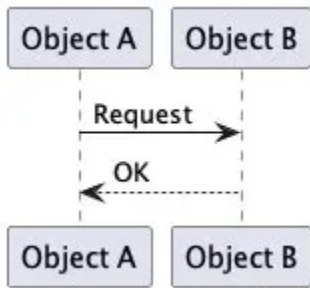
体验AI代码助手

复制代码

```

1 @startuml
2 participant "Object A" as A
3 participant "Object B" as B
4 A->>B: Request
5 return OK
6 @enduml

```



13、创建参与者

可以把关键字 **create** 放在第一次接收到消息之前，以强调本次消息实际上是在**创建**新的对象。

▼ shell

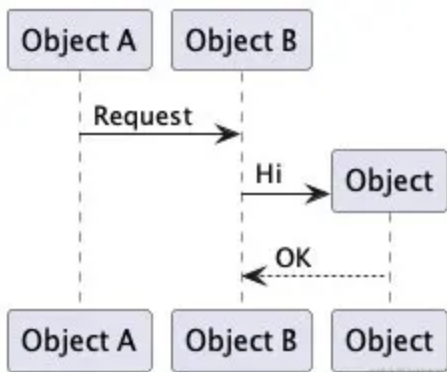
体验AI代码助手

复制代码

```

1 @startuml
2 participant "Object A" as A
3 participant "Object B" as B
4 A->>B: Request
5 create Object as C
6 B->>C: Hi
7 return OK
8 @enduml

```



14、锚点和持续时间

▼ shell

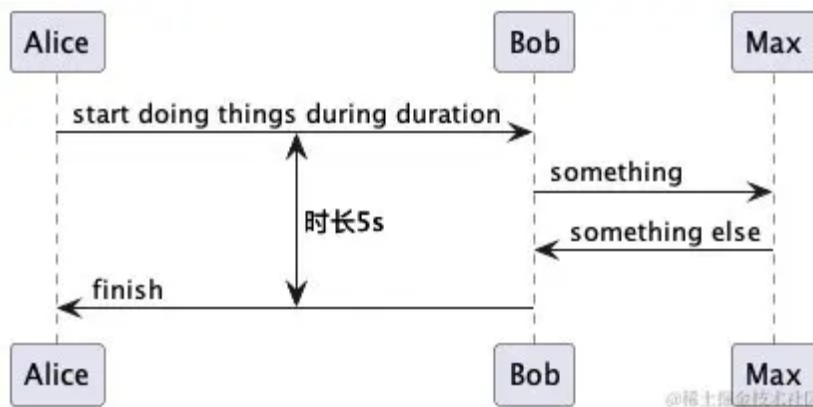
体验AI代码助手

复制代码


```

1  @startuml
2  !pragma teoz true
3
4  {start} Alice -> Bob : start doing things during duration
5  Bob -> Max : something
6  Max -> Bob : something else
7  {end} Bob -> Alice : finish
8
9  {start} <-> {end} : 时长5s
10
11 @enduml

```



15、构造类型和圈点

可以使用 `<<` 和 `>>` 给参与者添加构造类型。

在构造类型中，你可以使用 `(X,color)` 格式的语法添加一个圆圈圈起来的字符。

▼ shell

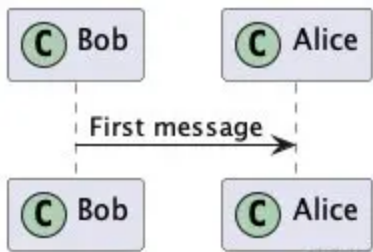
体验AI代码助手

复制代码

```

1  @startuml
2
3  participant Bob << (C,#ADD1B2) >>
4  participant Alice << (C,#ADD1B2) >>
5
6  Bob->>Alice: First message
7
8  @enduml

```



16、包裹参与者

我们可以使用 `box` 与 `end box` 生成一个盒子将**「参与者」**包裹起来，如：

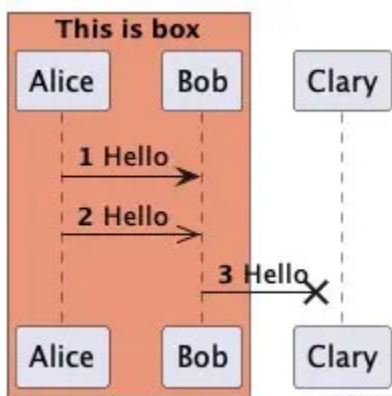
注意包裹的对象是参与者

▼ shell

体验AI代码助手

复制代码

```
1 @startuml
2 box "This is box" #DarkSalmon
3 participant "Alice" as A
4 participant Bob as B
5 end box
6 participant Clary as C
7 autonumber
8 A-> B: Hello
9 A->>B: Hello
10 B-x C: Hello
11 @enduml
```



三、外观参数

Skinparam 进阶

我们可以使用[skinparam](#)改变字体和颜色。可以在如下场景中使用：

- 在代码的全局定义中
- 在引入的文件中
- 在命令行或者ANT任务提供的配置文件中

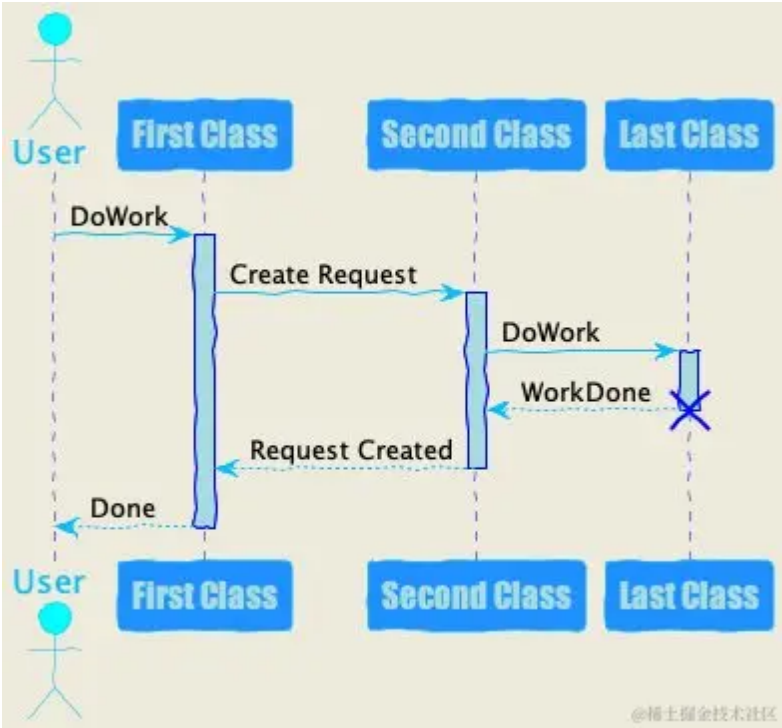
▼ shell

 体验AI代码助手 

复制代码

```
1 @startuml
2 skinparam backgroundColor #EEEBDC
3 skinparam handwritten true
4
5 skinparam sequence {
6 ArrowColor DeepSkyBlue
7 ActorBorderColor DeepSkyBlue
8 LifeLineBorderColor blue
9 LifeLineBackgroundColor #A9DCDF
10
11 ParticipantBorderColor DeepSkyBlue
12 ParticipantBackgroundColor DodgerBlue
13 ParticipantFontName Impact
14 ParticipantFontSize 17
15 ParticipantFontColor #A9DCDF
16
17 ActorBackgroundColor aqua
18 ActorFontColor DeepSkyBlue
19 ActorFontSize 17
20 ActorFontName Aapex
21 }
22
23 ' 正式开始
24
25 actor User
26 participant "First Class" as A
27 participant "Second Class" as B
28 participant "Last Class" as C
29
30 User -> A: DoWork
31 activate A
32
33 A -> B: Create Request
34 activate B
35
36 B -> C: DoWork
37 activate C
38 C --> B: WorkDone
39 destroy C
40
```

```
41 B --> A: Request Created
42 deactivate B
43
44 A --> User: Done
45 deactivate A
46
47 @enduml
```



标签：UML 后端

话题：1024一起掘金

本文收录于以下专栏



后端 专栏目录

Spring与服务器相关

1 订阅 · 17 篇文章

订阅

上一篇 理解 Go 中的切片：append ...

下一篇 2025年时序数据库发展方向和...

评论 0



[登录 / 注册](#) 即可发布评论!

暂无评论数据

目录

[收起 ^](#)

- 一、基本例子
- 二、基本语法
 - 1、方向
 - 2、参与者
 - 3、skinparam初阶
 - 4、箭头样式
 - 5、序号
 - 6、页面配置
 - 7、分组
 - 8、备注信息
 - 9、分隔符
 - 10、延迟
 - 11、生命线
 - 12、return
 - 13、创建参与者
 - 14、锚点和持续时间
 - 15、构造类型和圈点
 - 16、包裹参与者

三、外观参数

精选内容

换掉Navicat！一款集成AI功能的数据库管理工具，功能真心强大！

MacroZheng · 511阅读 · 3点赞

你不能直接用现成的吗？整个前端做笔记管理工具真是折腾人

四叶草会开花 · 352阅读 · 4点赞

想让AI更强大？ MCP协议了解一下！

程序员飞鱼 · 27阅读 · 2点赞

基于 RuoYi-Vue-Pro 定制了一个后台管理系统， 开源出来！

勇哥Java实战 · 23阅读 · 0点赞

Java volatile 关键字到底是什么 | 得物技术

得物技术 · 249阅读 · 7点赞

为你推荐

写设计方案时的绘图工具--PlantUML工具介绍

Eric223 2年前 👁 7.5k 👍 6 💬 1 后端 UML

技术分享：深入了解 PlantUML

FlyCodeLab 20天前 👁 238 👍 2 💬 评论 后端 面试 架构

UML 绘制工具 starUML 入门介绍

老马啸西风 1年前 👁 616 👍 点赞 💬 评论 Java

【程序员小知识】使用 PlantUML 画 UML（上）类图

fundroid 3年前 👁 4.6k 👍 29 💬 7 Android UML Java

PlantUML 全面指南

whysqwhw 1月前 👁 440 👍 点赞 💬 评论 编程语言

UML 架构图入门介绍 starUML

老马啸西风 1年前 👁 277 👍 点赞 💬 评论 Java

PlantUML 是绘制 uml 的一个开源项目

老马啸西风1年前👁 362👍 2💬 评论

Java

使用代码高效地生成UML图

来点vc11月前👁 718👍 2💬 评论

前端

绘图工具 draw.io / diagrams.net 免费在线图表编辑器

老马啸西风1年前👁 285👍 点赞💬 评论

Java

UML类图介绍

是王菜菜呀3年前👁 355👍 2💬 评论

后端

第1章 UML，提升软件开发效率与团队协作的可视化语言建模语言

TechLearn7月前👁 1.3k👍 4💬 2

UML后端架构

jsPlumb初体验

爆米花蘸粑粑3年前👁 4.2k👍 5💬 1

前端

可视化引擎Antv G6初体验（一） | 青训营

TIFFANY1年前👁 2.4k👍 6💬 评论

青训营...前端

UML基础

fliter1年前👁 404👍 3💬 评论

后端UML前端

代码结构设计(mayfly-go开源项目)

大熊全栈技术分享2年前👁 2.0k👍 22💬 1

GoDjango